

BUSINESS REQUIREMENTS & SOLUTION DESIGN

Online Food Ordering App

Single-restaurant ordering system — Node.js + Angular modular monolith, cash-on-delivery v1, built for a solo developer.

VERSION 1 · GENERATED AS A LIVING REFERENCE DOCUMENT

Contents

00	Executive Summary
01	Business Requirements
02	Context Diagram
03	Design Diagram
04	Architectural Diagram
05	Solution Architecture
06	Database Design
07	Screen Requirements — Customer App
08	Screen Requirements — Admin App
09	Design Decisions
10	Non-Functional Requirements
11	Assumptions

00 • Executive Summary

This document defines the business requirements and technical solution design for a single-restaurant online food ordering application. The restaurant owner manages the entire menu and order lifecycle through an admin panel; customers browse the menu, place orders, and pay cash on delivery. The system is built as a Node.js + Angular modular monolith, sized for roughly 200 orders/day, and is intentionally scoped to a lean v1 with clearly identified features deferred to later phases.

ITEM	VALUE
Business model	Single restaurant, direct-to-customer ordering
Expected load	~200 orders/day (low concurrency)
Target timeline	1-2 months, solo developer
Payment (v1)	Cash on Delivery only — online UPI payment deferred to Phase 2
Delivery	Handled outside the app by restaurant's own means; no in-app tracking
Hosting	Render.com (app + Redis), AWS RDS (PostgreSQL)

01 · Business Requirements

1.1 Purpose

Replace manual phone-order taking with a self-service ordering app that lets customers browse the menu, place orders, and pay on delivery, while giving the restaurant owner a simple panel to manage the menu and fulfill orders without needing technical help.

1.2 Stakeholders

STAKEHOLDER	ROLE / INTEREST
Restaurant Owner (Admin)	Sole administrator; manages menu, fulfills and tracks orders, configures delivery settings
Customers	Browse menu, place and pay for orders (COD), track order status, manage saved addresses
Delivery staff	Outside system scope — receives order details verbally/manually from Admin; no app access

1.3 Functional Requirements (Summary)

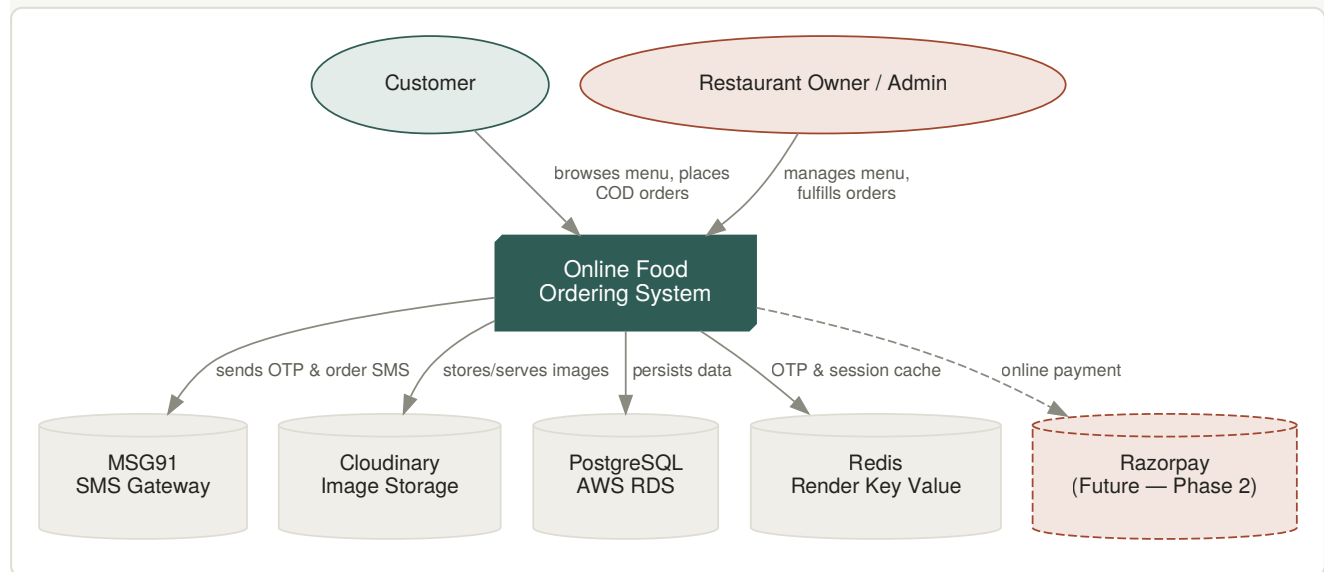
- **FR-1:** Customers can authenticate using mobile number + OTP, with no password and no email requirement.
- **FR-2:** Customers can browse menu items by category without logging in; login is required only when adding an item to cart.
- **FR-3:** Customers can maintain multiple saved delivery addresses, each validated against a configurable delivery radius from the restaurant.
- **FR-4:** Customers can place an order (COD) subject to a configurable minimum order value, and cancel it within 60 seconds of placement.
- **FR-5:** Customers can view ongoing and historical orders with current status.
- **FR-6:** Admin can add, edit, delete, and enable/disable (mark sold out) menu items, including image upload.
- **FR-7:** Admin can manage menu categories.
- **FR-8:** Admin can view incoming orders, update order status through the fulfillment lifecycle, and mark COD payment as collected.
- **FR-9:** Admin can toggle whether the restaurant is currently accepting orders.
- **FR-10:** Admin can configure the restaurant's location, delivery radius, and minimum order value — all without a code deployment.
- **FR-11:** Admin can view a basic sales report (order counts and revenue, daily/weekly).

1.4 Out of Scope for v1 (deferred to later phases)

Deliberately deferred: online payment gateway (Razorpay/UPI), discount coupons, scheduled/pre-orders, live delivery tracking, in-app delivery-partner communication, multi-restaurant/marketplace support, ratings & reviews, GST/tax computation, FSSAI & privacy policy display. Each is additive and does not require reworking the v1 data model.

02 · Context Diagram

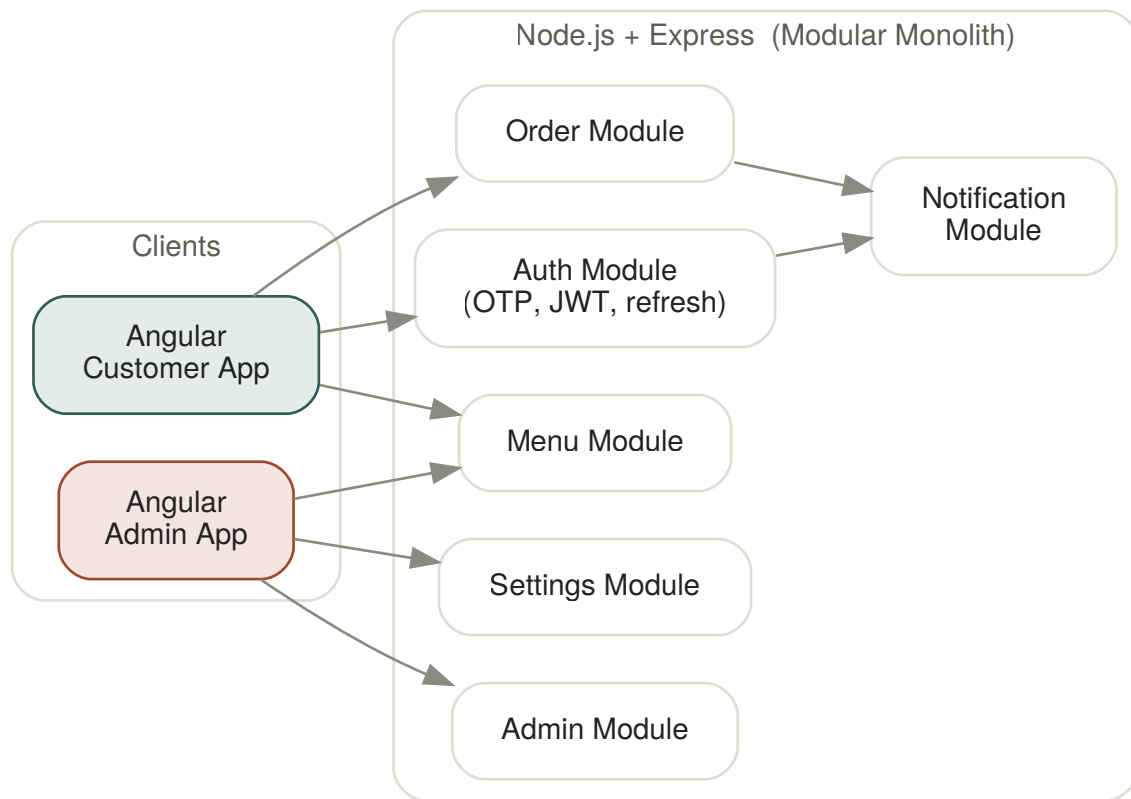
Shows the system boundary, its human actors, and the external services it depends on.



Dashed edge denotes a Phase 2 dependency, not present in v1.

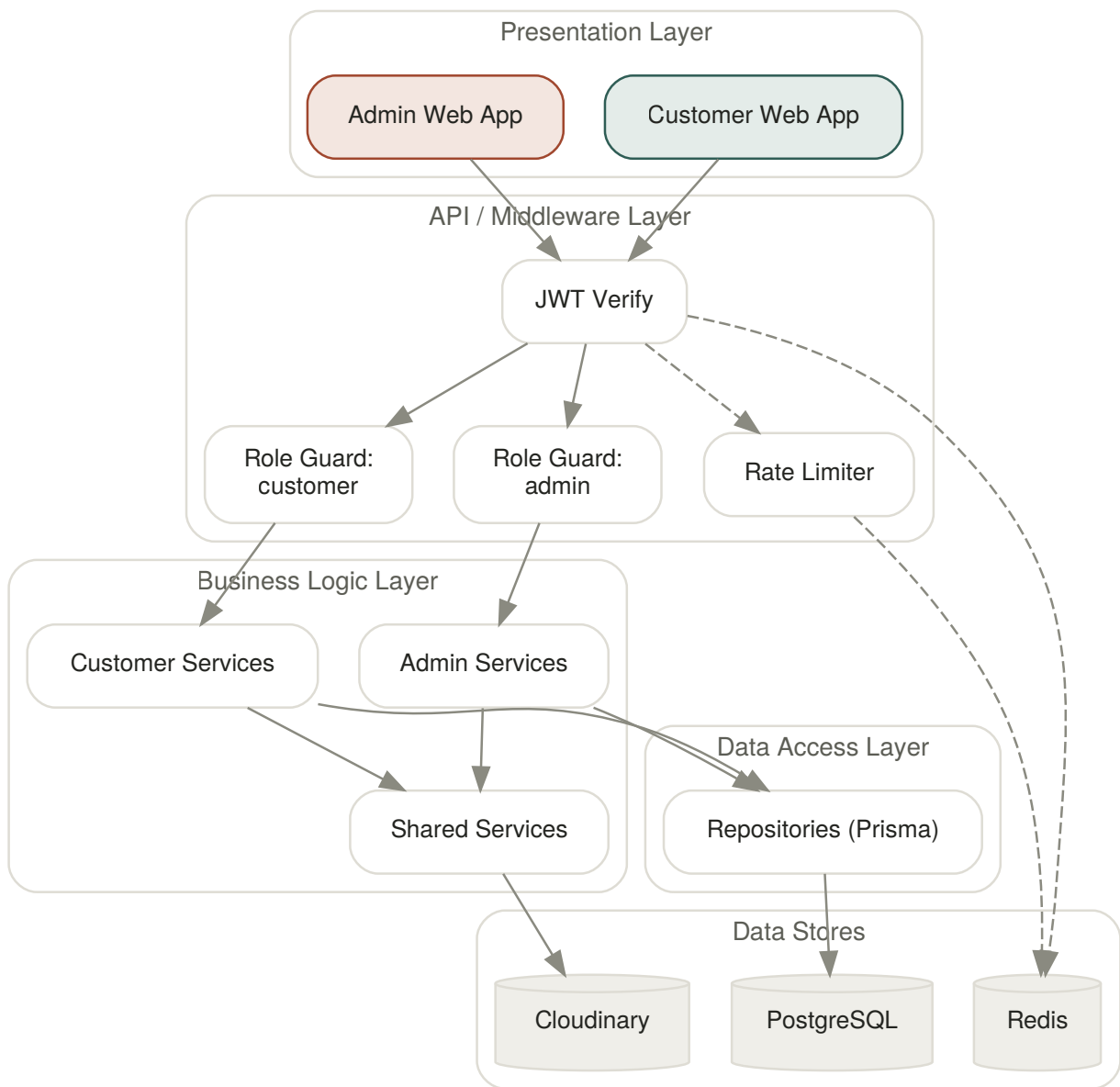
03 · Design Diagram

High-level component view — the two Angular front-ends, the single Node.js backend, and its modular internal boundaries.



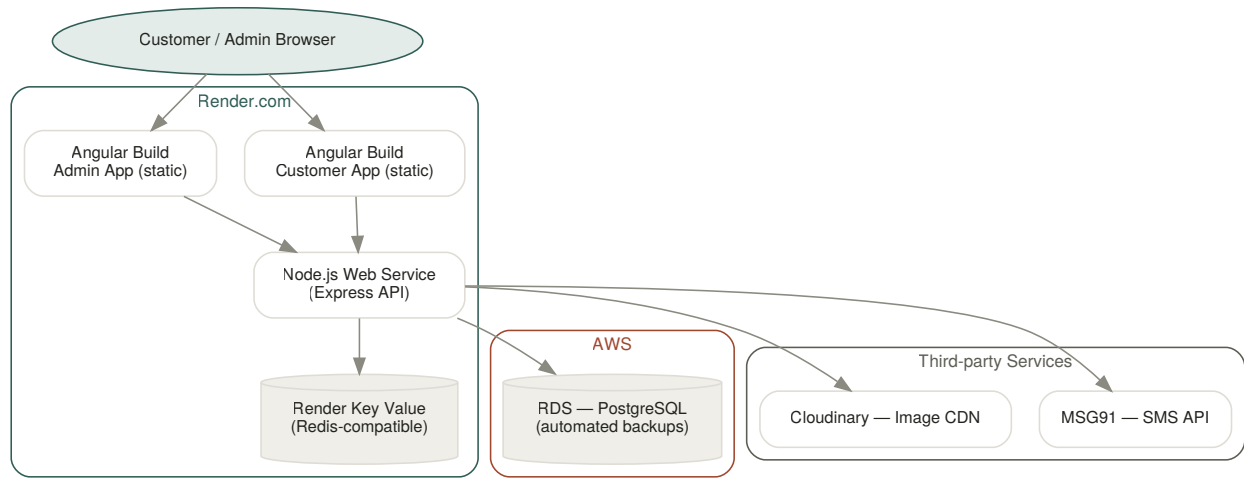
04 • Architectural Diagram

Logical layering, showing how the customer/admin separation is enforced at the middleware layer — not just hidden in the UI.



05 • Solution Architecture

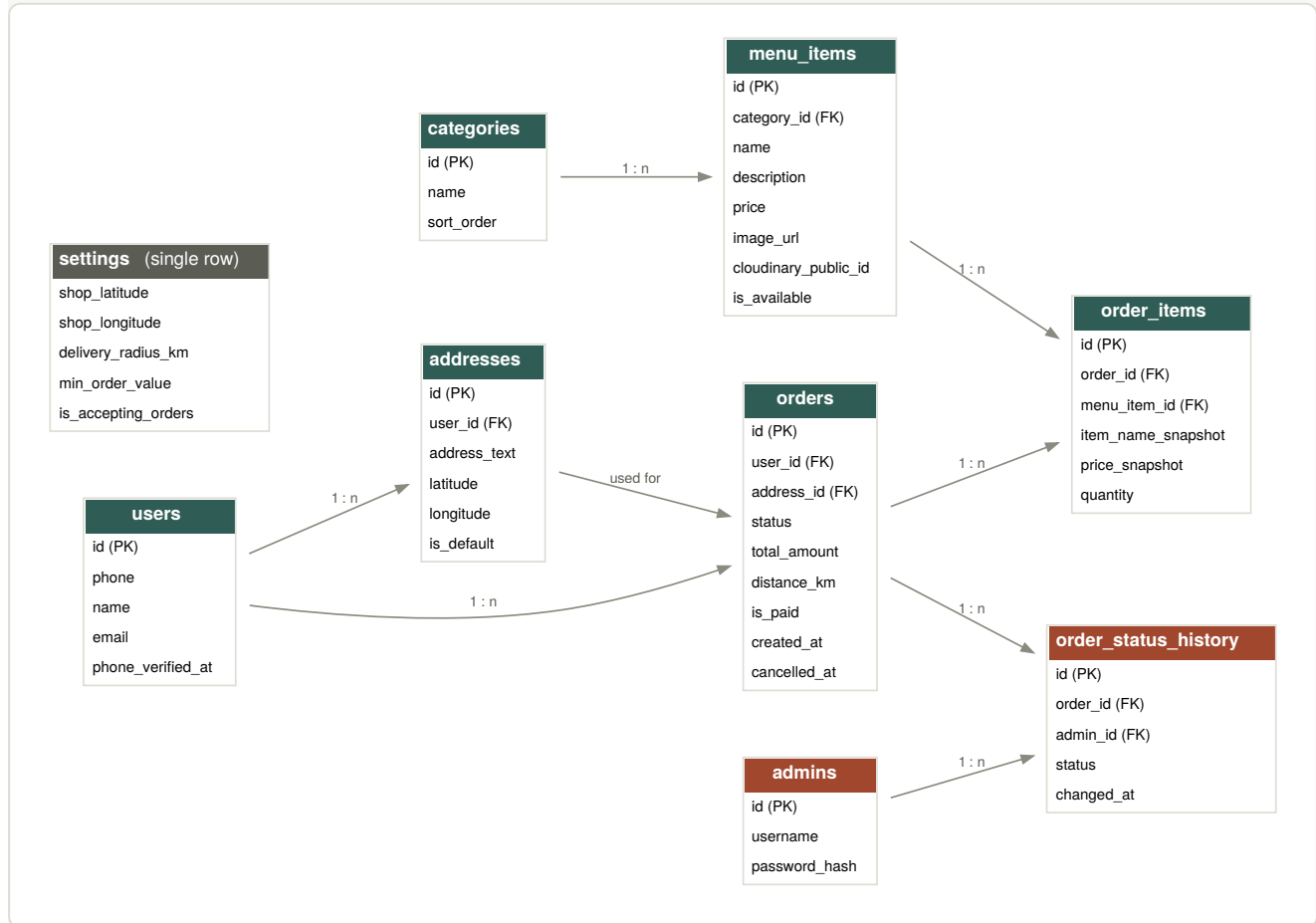
Deployment / infrastructure view — where each piece actually runs.



Single Node.js web service process hosts the entire modular-monolith API; Angular customer and admin apps are built and deployed as separate static sites, both calling the same backend under distinct route prefixes.

06 • Database Design

PostgreSQL holds all durable, relational data. Redis holds only ephemeral, TTL-based authentication data (not shown as relational tables).



6.1 Field-level detail

users

FIELD	TYPE	CONSTRAINTS / NOTES
id	uuid	PK
phone	string	unique, indexed — sole login identifier
name	string	required, collected on first login
email	string	nullable, optional, collected on first login, used only for receipts
phone_verified_at	timestamp	set on first successful OTP verification
created_at	timestamp	

admins

FIELD	TYPE	CONSTRAINTS / NOTES
id	uuid	PK
username	string	unique

password_hash	string	bcrypt; no recovery flow in v1 — see Assumptions
created_at	timestamp	
addresses		
FIELD	TYPE	CONSTRAINTS / NOTES
id	uuid	PK
user_id	uuid	FK → users.id
address_text	string	free-text / geocoded label
latitude / longitude	decimal(9,6)	captured via current-location or dragged map pin
is_default	boolean	only one true per user — enforce via partial unique index
categories		
FIELD	TYPE	CONSTRAINTS / NOTES
id	uuid	PK
name	string	e.g. Breakfast, Lunch, Dinner, Beverages
sort_order	int	controls tab/display order on menu
menu_items		
FIELD	TYPE	CONSTRAINTS / NOTES
id	uuid	PK
category_id	uuid	FK → categories.id
name, description	string, text	
price	decimal(10,2)	never float — exact currency math
image_url	string	Cloudinary delivery URL
cloudinary_public_id	string	needed to delete the asset on item delete/replace
is_available	boolean	false = "sold out"; item stays, hidden/greyed on customer menu
orders		
FIELD	TYPE	CONSTRAINTS / NOTES
id	uuid	PK
user_id	uuid	FK → users.id
address_id	uuid	FK → addresses.id (reference only)
delivery_address_text, delivery_latitude, delivery_longitude	string, decimal, decimal	snapshotted at order time so later address edits don't alter history
status	enum	PLACED · CONFIRMED · PREPARING · OUT_FOR_DELIVERY · DELIVERED · CANCELLED

total_amount	decimal(10,2)	
distance_km	decimal(6,2)	Haversine result at order time, snapshotted — not recalculated later
is_paid	boolean	COD — marked true by admin on cash collection
created_at	timestamp	cancellation window measured from here
cancelled_at	timestamp	nullable
order_items		
FIELD	TYPE	CONSTRAINTS / NOTES
id	uuid	PK
order_id	uuid	FK → orders.id
menu_item_id	uuid	FK → menu_items.id (reference only)
item_name_snapshot, price_snapshot	string, decimal	duplicated at order time — menu price/ name changes later must never alter past orders
quantity	int	
order_status_history		
FIELD	TYPE	CONSTRAINTS / NOTES
id	uuid	PK
order_id	uuid	FK → orders.id
admin_id	uuid	FK → admins.id — who made the change
status	enum	the status transitioned to
changed_at	timestamp	
settings (single row)		
FIELD	TYPE	CONSTRAINTS / NOTES
shop_latitude, shop_longitude	decimal(9,6)	set via admin map picker
delivery_radius_km	decimal(5,2)	configurable, used in Haversine check
min_order_value	decimal(10,2)	default 50, configurable
is_accepting_orders	boolean	live toggle on admin dashboard
6.2 Redis key structures (ephemeral, not relational)		
KEY PATTERN	VALUE	TTL
otp:{phone}	{ otp_hash, attempt_count }	5 min
ratelimit:otp:{phone}	send count	10 min
refresh:{token_hash}	user_id	30 days
user:{id}:sessions	set of active token hashes	mirrors longest-lived member

07 · Screen Requirements — Customer App

- 1 Login
- 2 OTP
- 3 Profile Setup*
- 4 Home/Menu
- 5 Item Detail
- 6 Cart
- 7 Address
- 8 Checkout
- 9 Confirmation

*Profile Setup appears only on a customer's first-ever login.

01 · Login — Enter Mobile Number

Purpose: Single entry point for both signup and login — no separate forms.

UI elements: Country code (fixed +91 or selectable), 10-digit phone input, "Send OTP" button.

Business rules: Validate 10-digit numeric format client-side; rate-limit sends server-side (max 3 per phone per 10 min).

API: POST /auth/send-otp

02 · OTP Verification

Purpose: Confirm phone ownership and issue a session.

UI elements: 6-digit code input (auto-advance boxes), "Resend OTP in 00:30" countdown, Verify button.

Business rules: OTP expires in 5 minutes; max 5 wrong attempts before forcing a resend; on success, backend finds-or-creates the user by phone.

API: POST /auth/verify-otp → returns access + refresh token.

03 · First-Time Profile Setup

Purpose: Collect the minimum info needed to address and contact a brand-new customer. Shown once, only for new users.

UI elements: Name (required), Email (optional, "for order receipts" helper text), Continue button.

Business rules: Skippable for email only, not for name; returning users never see this screen again.

API: PATCH /customer/profile

04 · Home / Menu

Purpose: The app's primary screen — effectively "the restaurant," since there's only one. No restaurant-discovery layer.

UI elements: Category tabs (from categories, ordered by sort_order), item cards (image, name, price, short description), optional search/filter bar, cart icon with item-count badge, "closed" banner when is_accepting_orders = false.

Business rules: Browsable without login. Items with is_available = false show an "Unavailable" badge and cannot be added to cart. If the shop is closed, menu is still viewable but "Add to Cart" is disabled app-wide with an explanatory banner.

API: GET /menu/categories, GET /menu/items, GET /settings/public

05 · Menu Item Detail

Purpose: Show full item info and let the customer choose quantity before adding to cart.

UI elements: Large image, name, full description, price, quantity stepper, "Add to Cart" button.

Business rules: If the customer is not logged in, tapping "Add to Cart" redirects to Login (screen 01) first, then returns to this screen with the action completed automatically.

API: GET /menu/items/:id

06 · Cart

Purpose: Review and adjust items before checkout.

UI elements: Line items with image, name, price, qty +/-, remove icon; subtotal; "minimum order ₹50 not met" warning if under threshold; "Proceed to Address" button (disabled below minimum).

Business rules: Minimum order value is read from `settings.min_order_value`, never hardcoded.

API: GET `/settings/public` for min order value; cart itself is client-side state until checkout.

07 · My Addresses / Add-Edit Address

Purpose: Maintain saved delivery addresses, each validated against the delivery radius.

UI elements — list: Saved addresses as cards (label, text, edit/delete icons), default address indicator, "Add New Address" button.

UI elements — add/edit: Map with draggable pin, "Use current location" button, address text / street / postcode fields, label chips (Home/Work/Other), "Save Location" button.

Business rules: On save, backend computes Haversine distance from `settings.shop_latitude/longitude`; if it exceeds `delivery_radius_km`, reject with a clear "outside our delivery area" message before saving. Only one address may be `is_default = true`.

API: GET `/customer/addresses`, POST `/customer/addresses`, PATCH `/customer/addresses/:id`, DELETE `/customer/addresses/:id`

08 · Checkout / Place Order

Purpose: Final review and order submission.

UI elements: Selected delivery address (editable), order summary (items, subtotal, "Free delivery" note, total), payment method shown as "Cash on Delivery" (fixed, non-editable in v1), "Place Order" button.

Business rules: Backend re-validates: shop is accepting orders, minimum order value met, delivery address within radius (re-checked, not trusted from the address-save step alone), all items still available.

API: POST `/customer/orders`

09 · Order Confirmation

Purpose: Confirm the order was placed and offer the brief cancellation window.

UI elements: Order ID, items summary, a visible 60-second countdown, "Cancel Order" button (disappears when the countdown ends).

Business rules: Cancel is only accepted server-side while `status = PLACED` and `now - created_at ≤ 60s` — the countdown is a UX aid, not the actual enforcement.

API: POST `/customer/orders/:id/cancel`

10 · My Orders (Ongoing / History)

Purpose: Let customers check on active orders and review past ones.

UI elements: Two tabs — Ongoing (`status ≠ DELIVERED/CANCELLED`) and History; each row shows item count, total, current status badge.

Business rules: No live map tracking — status is a simple badge updated by admin action, refreshed on screen focus or manual pull-to-refresh.

API: GET `/customer/orders?status=ongoing|history`

11 · Order Detail

Purpose: Full detail of one order.

UI elements: Items, prices, delivery address used, status timeline (derived from `order_status_history`), Cancel button only if still within window.

API: GET `/customer/orders/:id`

12 · Profile

Purpose: View/edit personal info; logout.

UI elements: Phone (read-only), Name (editable), Email (editable, optional), Logout button.

API: GET /customer/profile, PATCH /customer/profile

08 · Screen Requirements — Admin App

A1 · Admin Login

Purpose: Separate, simple authentication for the sole admin — no OTP, no OAuth.

UI elements: Username, password, "Log In" button.

Business rules: Issues an admin-scoped JWT (role: 'admin') with no overlap in route access with customer tokens. No self-service password reset in v1 — see Assumptions.

API: POST /admin/auth/login

A2 · Admin Dashboard Home

Purpose: At-a-glance operational view when the admin opens the app.

UI elements: "Accepting Orders" live toggle (prominent), today's order count, pending/unfulfilled count, simple revenue chart (daily/weekly), shortcut to Orders Queue.

Business rules: The toggle writes directly to settings.is_accepting_orders and takes effect immediately for all customers browsing the menu.

API: GET /admin/dashboard/summary, PATCH /admin/settings

A3 · Orders Queue

Purpose: Primary working screen for fulfilling orders.

UI elements: List of orders filterable by status, each row showing order ID, customer name/phone, item count, total, time since placed.

Business rules: Frontend polls GET /admin/orders?since=<timestamp> every 10 seconds — no WebSocket/real-time push in v1. New orders should be visually distinct until first opened.

API: GET /admin/orders

A4 · Order Detail (Admin)

Purpose: Fulfill a single order end-to-end.

UI elements: Items, customer name/phone, delivery address + distance, status dropdown/buttons (Confirm → Preparing → Out for Delivery → Delivered), "Mark Cash Collected" toggle, Reject option.

Business rules: Every status change writes a row to order_status_history with the acting admin's ID. Rejecting an order sets status = CANCELLED with a reason.

API: PATCH /admin/orders/:id/status, PATCH /admin/orders/:id/payment

A5 · Menu Management — My Food List

Purpose: See and manage all menu items at a glance.

UI elements: Category tabs, item rows (image, name, category tag, price, Available/Sold-Out badge), per-item "..." menu (Edit / Delete / Toggle availability), floating "+" to add a new item.

Business rules: Toggling availability is a single tap, no full-form reopen.

API: GET /admin/menu/items, PATCH /admin/menu/items/:id/availability, DELETE /admin/menu/items/:id

A6 • Add / Edit Menu Item

Purpose: Create a new dish or edit an existing one — same form, pre-filled when editing.

UI elements: Item name, category dropdown, photo upload (→ Cloudinary), price, description textarea, "Available for ordering" toggle (default on), Save button.

Business rules: On edit with a new photo, the old Cloudinary asset is deleted using its stored `cloudinary_public_id` before the new one is uploaded.

API: POST `/admin/menu/items`, PATCH `/admin/menu/items/:id`

A7 • Manage Categories

Purpose: Add, rename, reorder, or remove menu categories.

UI elements: Draggable/sortable list of categories, inline rename, add-new input, delete icon (blocked with a warning if items still reference it).

API: GET `/admin/menu/categories`, POST `/admin/menu/categories`, PATCH `/admin/menu/categories/:id`, DELETE `/admin/menu/categories/:id`

A8 • Settings

Purpose: Configure everything that would otherwise be hardcoded.

UI elements: Map picker for shop location (lat/long), delivery radius (km) input, minimum order value input, "Accepting Orders" toggle (mirrors the dashboard toggle).

Business rules: Changing shop location or radius does not retroactively affect `distance_km` already snapshotted on past orders.

API: GET `/admin/settings`, PATCH `/admin/settings`

A9 • Sales Report

Purpose: Basic business visibility — orders and revenue over time.

UI elements: Date range selector (today / this week / this month), total orders, total revenue, simple line/bar chart, optionally a top-selling-items list.

API: GET `/admin/reports/sales?from=&to=`

09 • Design Decisions

DECISION	RATIONALE
Modular monolith, not microservices	200 orders/day is roughly one order every 5–7 minutes at peak — no concurrency problem microservices would solve, and a single deployable is far easier for a solo developer to operate.
Phone + OTP login, no password, no OAuth	Removes password-reset flows and Google Cloud Console setup entirely; matches the pattern customers already expect from Swiggy/Zomato-style apps.
Email optional, collected (not required) at first login	Keeps signup friction minimal while still allowing receipts for customers who want them.
Redis for OTP and refresh tokens, not Postgres tables	Native TTL means no cleanup cron job; rate-limiting via INCR+EXPIRE is simpler than equivalent SQL.
COD-only for v1, Razorpay deferred	Removes payment-gateway integration, webhook signature verification, and refund handling from the critical path to launch.
Coupons and scheduled orders deferred	Each is additive (new table or nullable column) — deferring them reduces v1 scope without creating rework later.
Admin/customer strict separation	Separate Angular modules, separate JWT role claims, and backend middleware enforcement — a customer token can never reach admin routes regardless of what the UI allows.
Polling instead of WebSockets for admin order queue	At this order volume, a 10-second poll is operationally simpler than maintaining a Socket.IO connection.
Radius-based delivery check via Haversine, with lat/long snapshot on order	More accurate than a pincode list; snapshotting distance at order time keeps historical orders stable even if shop location or radius setting changes later.
Cloudinary for menu images	Render's web service has no persistent disk — images need external storage regardless.
Price/name snapshot on order_items	Historical orders must reflect what the customer actually paid, even after later menu price changes.

10 • How the Design Addresses Non-Functional Requirements

NFR	REQUIREMENT	HOW IT'S ADDRESSED
Performance	Responsive experience at ~200 orders/day	Low concurrency load easily handled by a single Node.js process; no caching layer needed beyond Redis for auth data.
Security	Protect customer data and prevent unauthorized access	Hashed OTPs, rate-limited OTP sends, JWT role claims checked by backend middleware, strict admin/customer route separation, parameterized queries via ORM.
Scalability	Room to grow without a rewrite	Modular monolith with clear module boundaries can be split into services later; Postgres and Redis are both independently scalable.
Reliability / Availability	Minimize downtime and data loss	AWS RDS automated backups; Render's managed deploys handle SSL and restarts; Redis data loss on restart only forces re-login.
Usability	Simple enough for a non-technical restaurant owner to run day-to-day	Single-tap availability toggle, one accept-orders switch, same Add/Edit form for menu items, minimal required fields throughout.
Maintainability	Easy for a solo developer to extend	Clear module-per-domain folder structure; deferred features designed to be additive, not disruptive.
Cost-efficiency	Fit a solo-developer budget	Free/low tiers throughout: Render + free Redis tier, Cloudinary free tier, MSG91 pay-per-SMS, no payment-gateway fees in v1.
Data integrity	Historical accuracy of orders despite later changes	Snapshotting (item price/name, delivery address, distance) at the moment of order placement.

11 • Assumptions

- Single restaurant, single admin user — no multi-staff roles or permission levels in v1.
- Delivery is carried out entirely outside the app; the system has no visibility into delivery-partner location or ETA.
- A customer's phone number is their sole account identifier — losing access to that number means losing account access, with no email-based recovery.
- Admin login has no password-recovery flow in v1; the owner is expected to store the admin password securely.
- Cash on Delivery is the only payment method in v1 — reconciliation is manual, via admin marking `is_paid`.
- No GST/tax computation is applied to order totals in v1.
- FSSAI license display and a Privacy Policy page are deferred past v1 launch.
- Secrets are currently planned to be stored in a `.env` file; this document flags the risk of credential leakage if committed to a shared repository, and recommends Render's environment-variable dashboard as a same-effort, safer alternative.
- Cloudinary's and Render's free tiers are assumed sufficient at this scale; revisit if order volume or catalog size grows materially.
- No automated test suite is assumed beyond integration coverage for order placement and cancellation, given the solo-developer, 1-2 month timeline.